



PERSONAL PROJECT | DATA AUTOMATION

Daily FX Rates

From a public FX rates source to a live report, with no manual steps

Sergio / October 2026

Why I Built It

It started as a proof of concept: using Visual Studio Code with AI to build something real.

It was also a project that could grow: getting the FX rates I need in my other reports, to convert data into a base currency such as USD.

1

Where can I get FX rate data for free?

2

How do I use an API to bring the data in?

3

My Python and SQL are intermediate. How far can AI help me go?

4

How do I make the whole flow run by itself?

What Was Done

A daily feed of EUR and GBP rates against USD, stored and reported automatically

1

A Python script gets the daily EUR and GBP rates against USD from a public rates source (the Frankfurter API).

2

The rates are saved in two places: a CSV file as a backup, and a PostgreSQL table in a Neon cloud database.

3

The job runs by itself every day at 18:00 UTC with GitHub Actions. A Slack message is sent if it fails.

4

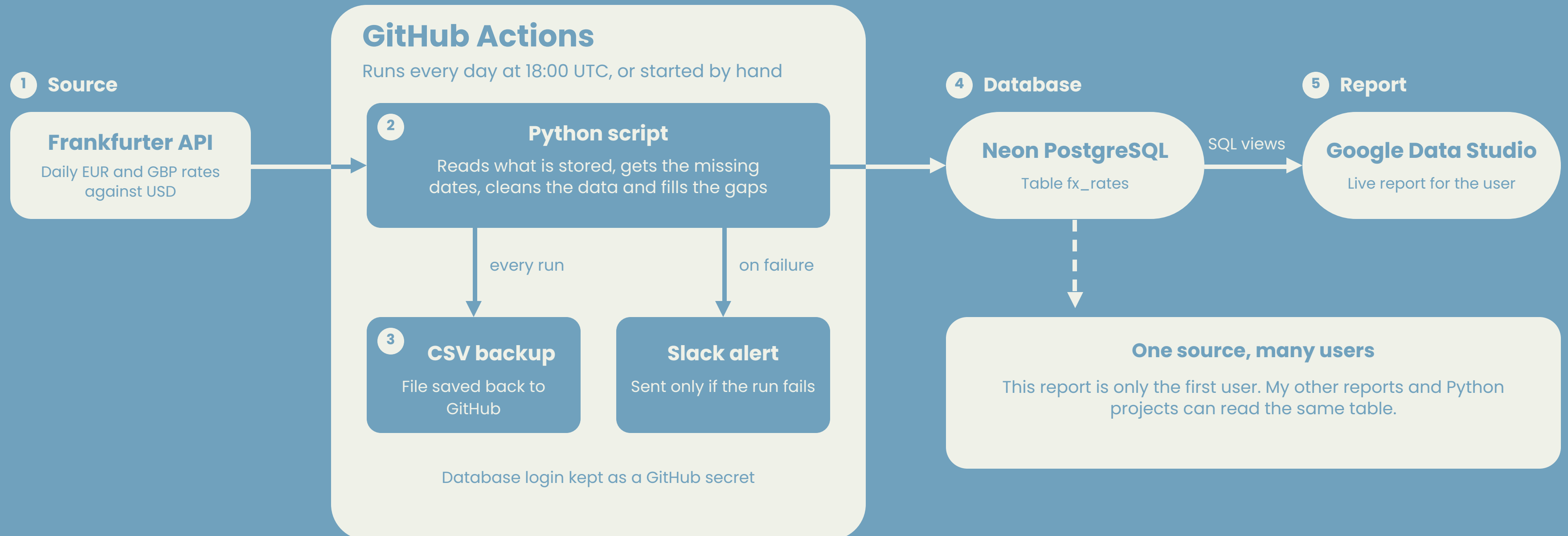
A Google Data Studio report reads the database and refreshes itself once a day. It shows GBP/USD, EUR/USD and GBP/EUR.

THE RESULT

One clean source of FX rates that my other reports and projects can use.

The goal: prove I could build this in Visual Studio Code with AI, and get the FX rates my other reports need.

How the Workflow Runs



1 Get rates → 2 Clean and fill gaps → 3 Save CSV backup → 4 Update database → 5 Report shows new rates

How I Worked with AI

I started with GitHub Copilot in Visual Studio Code. Now I use Claude.

My part

- The idea, and clear instructions for every step
- Testing and debugging
- Some pieces myself with AI assistance, like the table settings and the unique key
- The Data Studio report

AI's part

- Filling my gaps on the most technical side: the code and the connections
- Writing the code from my instructions
- Helping me solve each issue I found when testing
- Helping me polish the report

WHY DATA STUDIO AND NOT POWER BI

I can share a link and anyone can open the report. Power BI needs a licence and admin rights to share. I would have been quicker in Power BI, and the report would look nicer, but nobody outside could see it.

Decisions I Made

Four choices that shaped the project

What I decided	Why
USD as the base currency	My other reports need to convert data into one base currency. With every rate stored against USD, the conversion is always the same.
A rate for every day, also weekends and holidays	I learned this in Power BI: a day with no rate cannot be converted. Empty days use the last known rate and are flagged "filled"; market rates are flagged "raw".
A database, with a CSV file as backup	I started with only a CSV file. Then I moved to a database and found Neon, a free PostgreSQL option. The CSV stayed as the backup.
A full check once a week, not on every run	Checking the whole history every day was not needed. Daily runs look at the last 14 days. Every Monday the script checks everything.

Testing and Debugging

DAYS WITH NO DATA

Some days were missing from the data completely.

The fix

Every Monday the script checks the whole history for missing days and loads them again. Daily runs check the last 14 days.

DUPLICATE DATA

The same date and currency appeared more than once.

The fix

A unique key on date and currency. Existing rows are updated, not added again.

WRONG VALUES

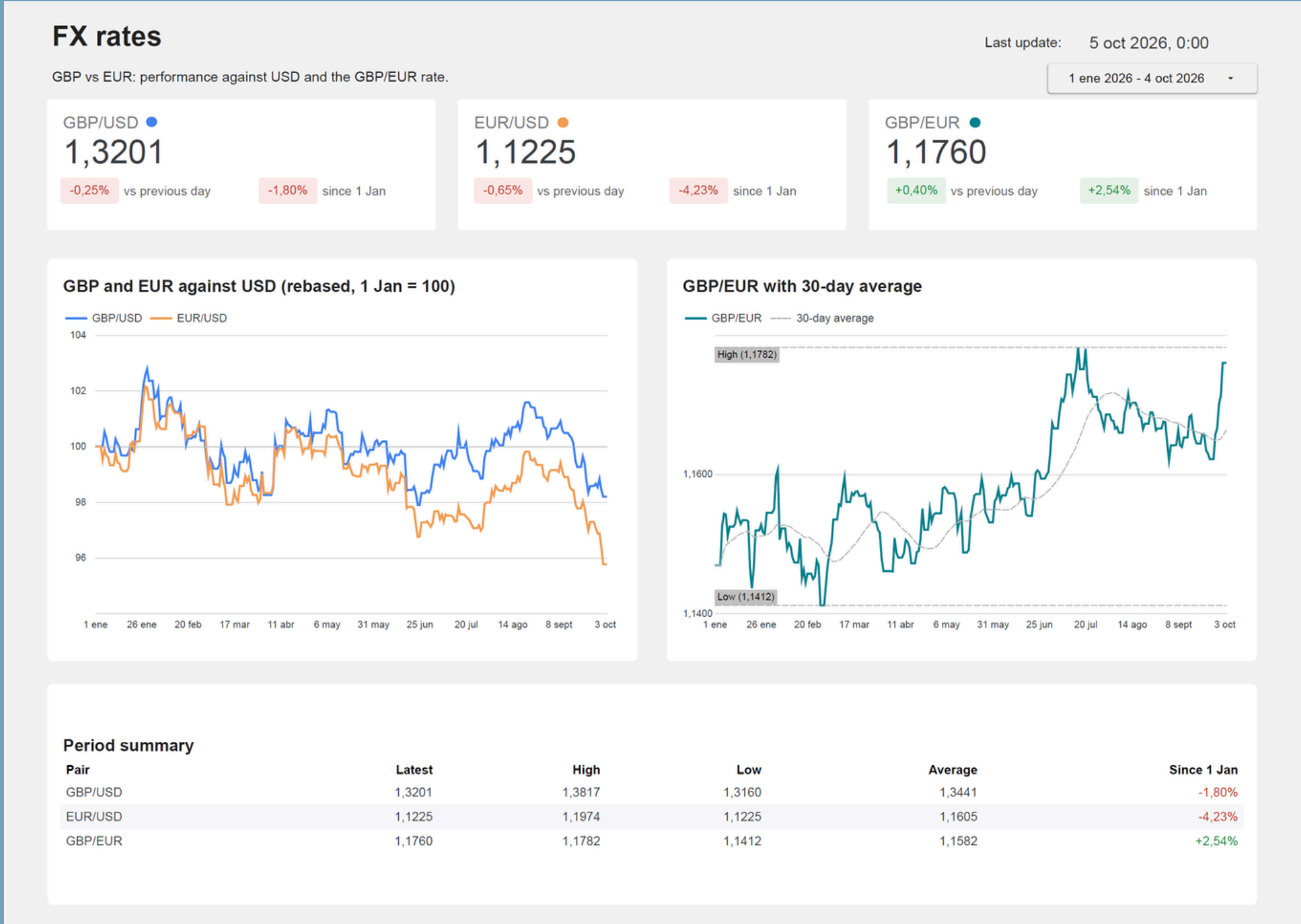
Some days were filled before the real rate arrived, so the value was wrong.

The fix

Filled rates from the last 7 days are checked again on every run, and replaced when the real rate arrives.

I log every issue in GitHub and then solve it. Getting the data was the easy part. Making it reliable was the real work.

The Final Result



3

FX rates in one live report

GBP/USD and EUR/USD come from the market. GBP/EUR is calculated from both.

0

manual steps from source to report

The job runs every day at 18:00 UTC and alerts me only if it fails. Data Studio refreshes the report once a day.

2

quality flags on every rate

Raw for a market rate, filled for a rate carried forward on a weekend or holiday.

See the live report: <https://datastudio.google.com/s/sC5tlwXPGVI>

Four Questions, Answered

AI helped me with my gaps on the most technical side: the code and the connections.

Testing, debugging and the decisions stayed with me.

1

Free FX data: the Frankfurter API, a public source of daily rates at no cost.

2

Using an API: one request from a Python script brings the rates in, ready to clean and store.

3

AI: it wrote the code from my instructions and helped me solve each issue. I tested, debugged and decided.

4

Running by itself: GitHub Actions runs the script every day and alerts me if it fails. Data Studio refreshes the report once a day.